

Архитектура роутинга выплат

Путь одной операции внутри `Routing::Engine`, `Routing::Router` и `Routing::RuntimeState`. CLI не показан. CSV с историей на этот путь не влияет: он используется только в отчёте.

8

Проверок hard-constraints

7

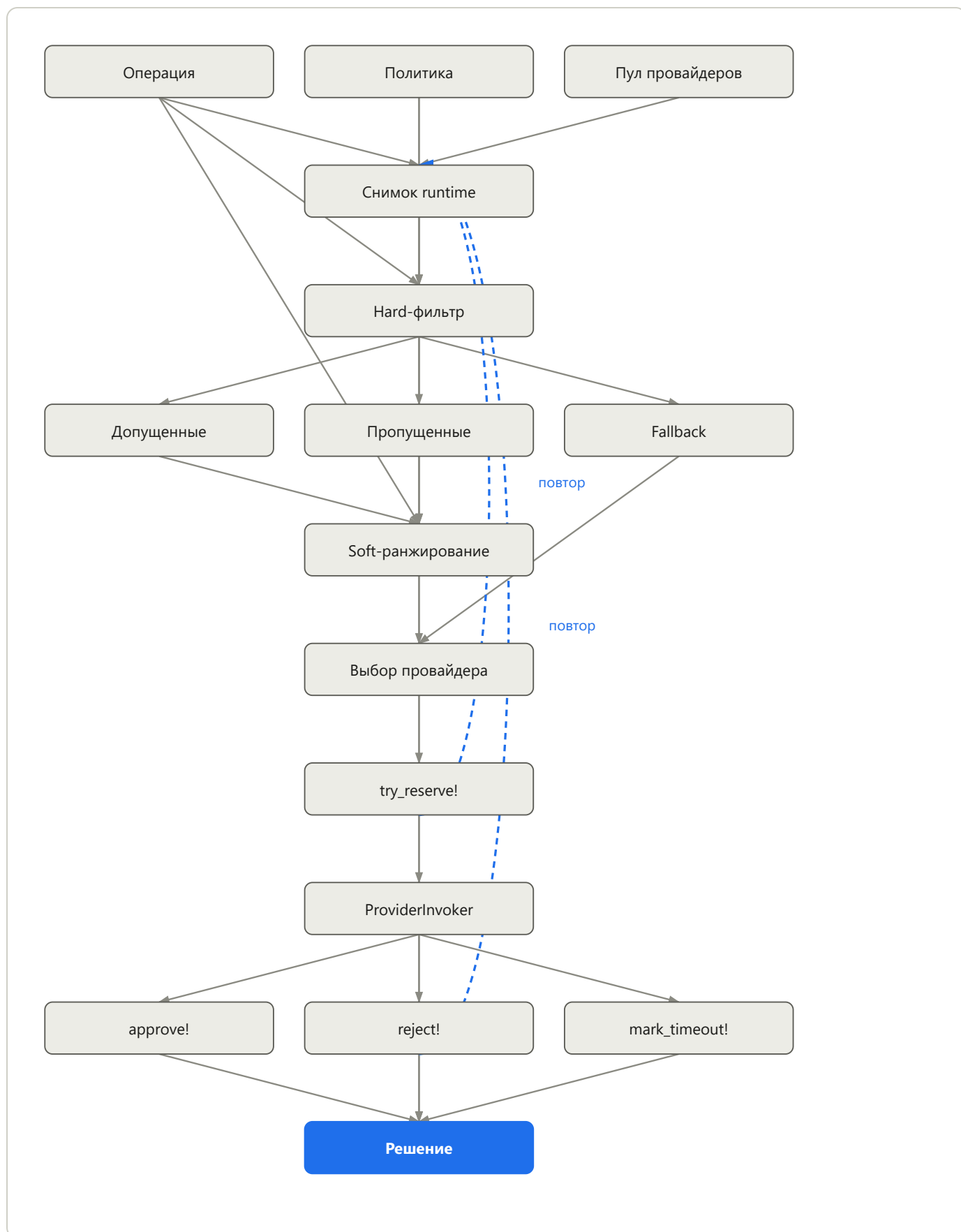
Оценщиков soft-goals

3

Исхода симуляции

Исполнение одной операции

Вызов `Engine#route_one` снимает снимок живого состояния провайдеров, допускает пул по hard-constraints, ранжирует его взвешенными soft-goals, резервирует ёмкость относительно той же ревизии снимка и симулирует провайдера. Явный отказ основного провайдера повторяет цикл с его именем в `attempted`. Таймаут удерживает reservation и останавливает каскад до status-check. Пунктир — повтор после отказа или устаревшей ревизии, а не второй параллельный путь.



Источник: lib/routing/engine.rb, router.rb, runtime_state.rb · одна операция, включая каскадные повторы. Сплошные рёбра — прямой поток; пунктир — повтор.

Что передаётся на каждом шаге

Откуда	Куда	Данные
RuntimeState#snapshot	Router	Замороженные копии провайдеров, SoftGoals::Snapshot (счётчики сессии + дневной объём), ревизия
HardConstraints::Filter	Ranker / Engine	допущенные основные провайдеры, пропуски, fallback или nil
SoftGoals::Ranker	Selector	упорядоченный список, Score по провайдеру (итог + вклады), конфликты, пометки о недостижимых целях
Selector / Filter	Engine	Selection: предпочтительный основной либо fallback, если ранжируемый пул пуст
Engine	RuntimeState#try_reserve!	Выбранный Provider, Operation, expected_revision
ProviderInvoker	Engine	{ result: approved rejected expired, latency_sec }
RouteContext	Decision	attempts[], накопленный latency_sec, selected_provider, simulated_result

Слои

Библиотека разделяет допуск, оценку, оркестрацию и состояние: новая проверка или цель — это класс в реестре, а не ветка в цикле попыток.

Слой	Типы	Ответственность
Входные сущности	Operation, Provider, ProviderPool, Policy	Типизированные записи из JSON/YAML. Политика хранит веса стратегий, профили провайдеров, имя fallback и зерно симуляции.
Допуск	HardConstraints::*	Первая непройденная проверка останавливает провайдера. Статус, диапазон суммы, дневной лимит, in-progress count/amount, банк, маржа, реквизиты, RPM.
Оценка	SoftGoals::*	Только допущенные основные провайдеры. Взвешенная сумма; ничья — по provider.priority, затем по имени. Fallback здесь не участвует.
Выбор	Router, Selector	Router вызывает Filter, затем Ranker, затем Selector. Selector возвращает ranking.preferred. Если он nil, Router отдаёт hard-допустимый fallback.
Оркестрация	Engine, RouteContext	Онлайн-инварианты (уникальный id, неубывающий created_at), цикл попыток, журнал attempts, сумма latency, сборка Decision.
Состояние	RuntimeState, Reservation, счётчики Provider	Mutex и ревизия. Резерв атомарный: повторная проверка hard-constraints на живом провайдере.

Hard-constraints

Бинарный допуск. Отказ пишет skipped-попытку и убирает провайдера из пула этой операции.

Проверка	Пропуск, если
Status	status != "active"
AmountRange	сумма вне limit_amount_min/max
DailyLimit	committed + reserved + amount > daily_amount_limit
InProgress	превышен лимит числа или суммы in-progress
BankFilter	банк не в allowlist или в exclude
Margin	provider_margin > merchant_margin без waiver
Requisites	available_requisites == 0
Intensity	запросов за последние 60 с ≥ RPM-лимита

Soft-goals

Каждый вклад в [-1, 1]. Политика умножает на вес профиля провайдера. Итоги сравнимы: веса поставленных профилей суммируются в 1.0.

Цель	Оценка
count_share	дефицит относительно traffic_percentage
volume_share	дефицит относительно volume_share_pct
conversion	conversion_24h в [0, 1]
cascade_priority	1 / priority
amount_band	amount / limit_amount_max: выше у более узкого подходящего потолка
financial_obligation	буст ниже min; штраф только от 80% soft-max
load_balance	1 – макс. утилизация in-progress и RPM

Резерв и счётчики

`try_reserve!` успешен, только если ревизия снимка ещё актуальна и живой провайдер по-прежнему проходит hard-constraints. Затем увеличиваются in-progress count/amount, daily_reserved_amount, pending-счётчик сессии и окно запросов за 60 секунд. `available_requisites` уменьшается на резерве и возвращается на release.

Исход	Счётчики	Цикл попыток
● approved	Снять in-progress и daily_reserved; зафиксировать daily_approved; pending → committed count	Стоп. Decision.selected_provider — этот провайдер.
● rejected	Снять in-progress и daily_reserved; обнулить pending; не коммитить оборот	Если fallback: стоп с simulated_result rejected. Иначе skip (simulated_rejected), имя в attempted, новый снимок.
● expired	Сохранить in-progress, daily_reserved и pending до позднего ответа; пометить reservation как timed_out	Временно считать успехом и остановить каскад. Fallback до status-check не запускать.

Правило выбора

Предпочтительный = наибольший взвешенный soft-score среди оставшихся допущенных основных провайдеров. Остальные допущенные попадают в `attempts` как `lower_soft_score`. Fallback (`spacpayments`) держится вне ранжирования и используется, только когда этот набор пуст. Устаревший резерв попытку не тратит: цикл снова снимает снимок с тем же `attempted`. Явный отказ исключает провайдера и переводит операцию к следующему. Таймаут удерживает reservation и завершает текущий каскад до позднего status-check.

Подключение стратегий

Смесь стратегий меняется только в `config/routing_policy.yml`. Прямое включение стратегий и режим профилей взаимно исключают друг друга. Поставляемый файл использует `active_profile: balanced` и `provider_profiles`: vipay, payflow и quickpay считаются разными векторами весов, затем сравниваются по итогу.

Стратегия из брифа	Реализация
Доля по числу заявок	count_share + committed/pending счётчики сессии
Доля по объёму	volume_share + daily_approved_amount (committed + reserved)
Каскадная очередь	cascade_priority, затем имя; повторы переранжируют оставшихся
По сумме платежа	hard AmountRange плюс amount_band (доля суммы от max — более узкий потолок)
По конверсии	цель conversion по conversion_24h
По интенсивности	hard Intensity (RPM); load_balance учитывает in-progress и RPM
Финансовые обязательства	financial_obligation по daily_turnover_min/max

Router.call

Оставшиеся провайдеры = пул минус `attempted`. Filter относит каждого к допущенным основным, пропущенным основным или fallback. Ranker считает только `evaluation.eligible`. Selector возвращает `ranking.preferred`. Engine резервирует этого провайдера либо fallback, если preferred равен nil. Допущенные, но не выбранные провайдеры записываются в attempts с причиной `lower_soft_score`.

Новая hard-проверка: добавить в `HardConstraints::CHECKS`. Новая soft-цель: добавить в `SoftGoals::GOALS` и ключ в YAML. Engine не ветвится по именам стратегий.